

Tex_Agent 软件 SBOM 与代码说明文档

团队成员：毛伟翔、于天泽、乔雨霖、唐俊涛

2026 年 6 月 12 日

目录

1	代码结构与模块依赖	2
1.1	主要目录结构	2
1.2	测试说明	2
1.3	模块依赖关系图	2
1.4	主要目录作用速查	3
2	软件 SBOM 清单	4
2.1	来源分类说明	4
2.2	大模型生成内容与 Prompt 溯源	5
2.3	人工加工与演进说明	6
2.4	按目录来源标注总表	6
2.5	第三方开源依赖清单	7
2.5.1	工作流与 LLM 框架	7
2.5.2	Web 服务	8
2.5.3	RAG 与向量检索	8
2.5.4	LaTeX 诊断	8
2.5.5	PDF 与文档解析	8
2.5.6	可视化、工具与测试	9
2.5.7	前端 vendor 开源库（非 pip）	9
3	项目概述	9
3.1	架构总览	9
3.2	快速启动	10
3.3	文档范围说明	10

目录	2
4 根目录文件	10
4.1 核心入口与配置	10
4.2 开发辅助脚本（以下划线开头）	12
4.3 其他根目录文件	12
5 agents/ 一智能体实现层	12
5.1 目录总体作用	12
5.2 文件说明	12
6 core/ 一核心基础设施	13
6.1 目录总体作用	13
6.2 文件说明	14
7 config/ 一配置层	15
7.1 目录总体作用	15
7.2 Python 模块	15
7.3 JSON 与数据文件	16
8 context/ 一上下文管理	16
8.1 目录总体作用	16
8.2 文件说明	17
9 workflow/ 一工作流引擎	17
9.1 目录总体作用	17
9.2 文件说明	17
9.3 节点类型	19
10 router/ 一任务规划与路由	19
10.1 目录总体作用	19
10.2 文件说明	19
10.3 规划流程	20
11 tools/ 一可插拔工具层	20
11.1 目录总体作用	20
11.2 基础设施	20
11.3 已注册工具（workflow 可用）	20
11.4 未注册 / Web 独立工具	22
12 latex/ 一LaTeX 子系统	22
12.1 目录总体作用	22

12.2 核心数据与契约	23
12.3 项目扫描与索引	23
12.4 解析与语法检查	23
12.5 诊断 (L1/L2) 与合并	24
12.6 切片、修复与建议	24
12.7 监视与幽灵窗口	25
12.8 环境与导出	25
12.9 LaTeX 诊断修复流水线	25
13 rag/ 一检索增强生成	26
13.1 目录总体作用	26
13.2 文件说明	26
14 memory/ 一记忆系统	27
14.1 目录总体作用	27
14.2 文件说明	27
15 ui/ 一用户界面层	27
15.1 目录总体作用	27
15.2 ui/web/ 一主 Web 服务	28
15.3 ui/web/static/ 一主 Web 前端	28
15.4 ui/overleaf/ 一LaTeX 编辑器	29
15.5 ui/ghost/ 一幽灵窗口	29
16 cli/ 一终端命令框架	30
16.1 目录总体作用	30
16.2 文件说明	30
17 utils/ 一通用工具	30
17.1 目录总体作用	30
17.2 文件说明	30
18 scripts/ 一启动脚本	31
18.1 目录总体作用	31
18.2 文件说明	31

1 代码结构与模块依赖

本节给出 TeX_Agent 主要目录的浅层结构树与模块依赖关系，层级深度控制在 1-2 级，便于从整体上把握系统构成（详细文件说明见后续各章）。

1.1 主要目录结构

```
TeX_Agent/  
|-- main.py / check.py / check_text.py / overleaf.py    # 程序入口  
|-- start_all.bat          # Windows 一键启动 Chat + Writing 双服务  
|-- config/               # 统一配置与 workflow 注册表  
|-- core/                 # 消息协议、状态、CLI 编排核心  
|-- workflow/            # LangGraph 工作流引擎  
|-- agents/              # 多智能体实现  
|-- tools/               # 可插拔工具层  
|-- context/ + memory/ + rag/  # 上下文 / 记忆 / 检索增强  
|-- router/              # Plan 模式任务规划  
|-- latex/               # LaTeX 诊断修复子系统  
|-- ui/                  # Web UI / Overleaf 编辑器 / Ghost 窗口  
|-- cli/ + utils/        # 终端命令框架 / 通用工具  
|-- scripts/             # 启动脚本  
`-- tests/ + test/       # 测试代码（详见测试文档）
```

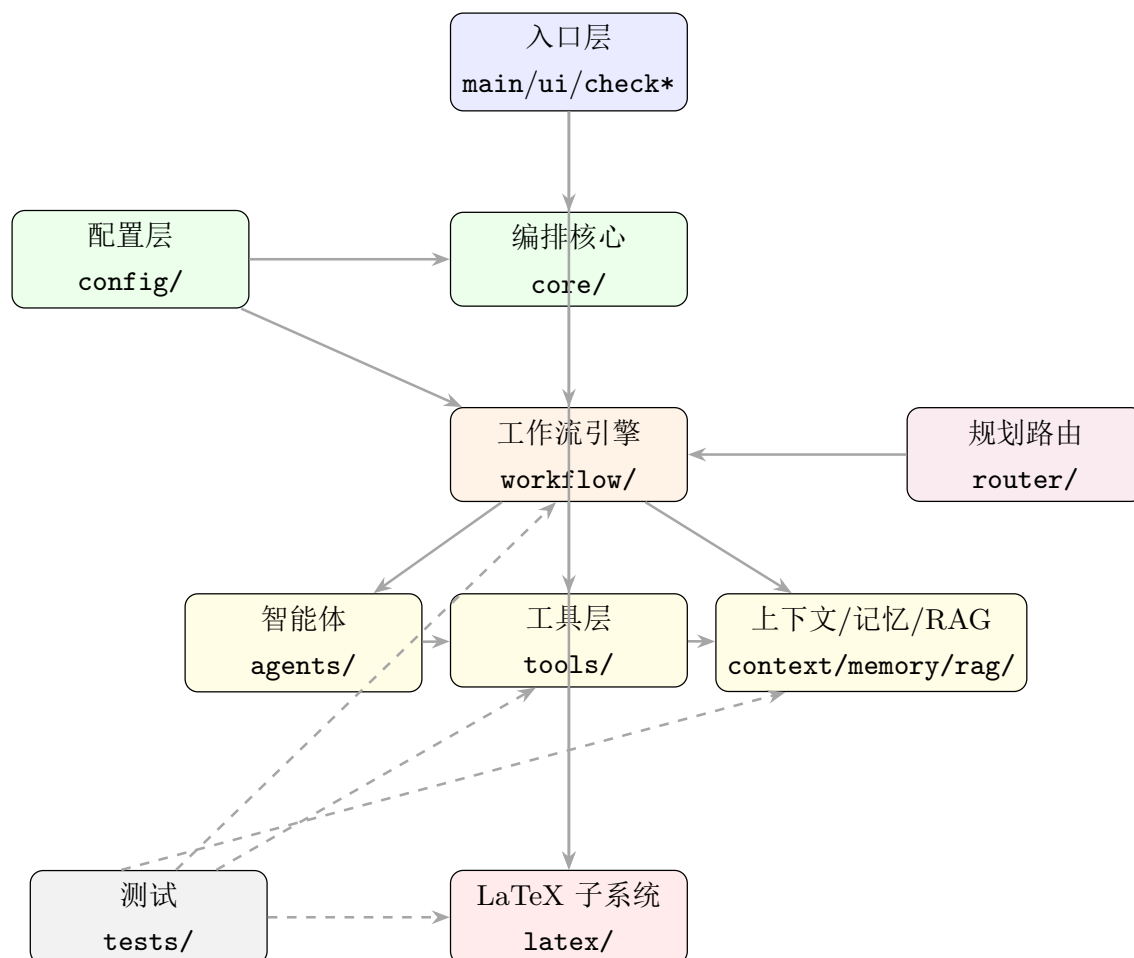
1.2 测试说明

项目提交了额外的测试文档，用例范围、目录组织、执行方式与结果等详情请参阅测试文档，本文档不再展开各测试文件说明。

测试代码主要位于仓库根目录 tests/；运行配置见 pytest.ini。

1.3 模块依赖关系图

下图展示各主要文件夹之间的调用/依赖方向（箭头表示「依赖或被调用」关系，非文件包含关系）。



读图说明：

- 入口层（main.py、ui/、check*.py）统一委托 core/agent_cli.py 编排执行。
- config/ 为全局配置源，被 core/、workflow/、agents/ 等读取。
- workflow/ 是执行中枢，调度 agents/、tools/，并借助 context/、memory/、rag/ 拼装 Prompt。
- router/ 仅在 Plan 模式下介入，将用户任务分解为动态图后交给 workflow/。
- latex/ 作为独立子系统，主要由 tools/latex_* 与 ui/ghost/ 调用。
- 虚线表示 tests/ 对各个功能模块的验证依赖（测试调用，非运行时依赖）。

1.4 主要目录作用速查

目录	作用	主要依赖
config/	环境变量、工作流 JSON、上下文 Profile、Planner Schema	无（被各层读取）
core/	消息协议、WorkflowState、TeX-AgentCLI 编排入口	config/, workflow/
workflow/	JSON/TaskPlan → LangGraph 图编译与执行	config/, agents/, tools/
agents/	LLM 智能体 (Simple/Multi/Reflection 等)	core/, tools/
tools/	可插拔工具 (arXiv、PDF、LaTeX、RAG 等)	latex/, rag/, memory/
context/	GSSC 上下文流水线，拼装 Agent Prompt	config/, memory/
memory/	分支记忆、用户画像持久化	core/
rag/	ChromaDB 向量检索管道	config/
router/	Plan 模式 MAS 任务分解与图生成	config/, workflow/
latex/	LaTeX 项目索引、L1/L2/L3 诊断修复、Ghost 窗口	系统 TeX 工具链 (chktex/latexmk)
ui/	Web 聊天 UI、Overleaf 编辑器、Ghost 浮层	core/, latex/, rag/
cli/	终端 REPL 命令注册与分支管理	core/
utils/	日志、并发、回复格式化、取消控制等	config/
tests/、test/	pytest 测试与演示脚本；详见测试文档	被测模块

2 软件 SBOM 清单

本节按软件物料清单 (SBOM) 要求，说明各模块的来源分类、大模型 Prompt 溯源、人工加工说明及第三方开源依赖。

2.1 来源分类说明

本项目代码来源分为四类：

自研 项目组成员在仓库中独立设计并实现的功能代码。

大模型生成

依据 doc/ 目录下 Prompt 文档及其他辅助说明性文字，由大语言模型生成的初始项目骨架与接口占位代码。

大模型辅助生成

在大模型建议或草稿基础上，由成员审核、测试、修改后纳入仓库的代码。

复用开源

通过 requirements.txt 引入的第三方 Python 库，或前端 vendor 目录中的开源 JS 库。

2.2 大模型生成内容与 Prompt 溯源

生成内容	Prompt / 设计文档	说明
系统总体目录结构与初始框架骨架	doc/代码框架要求.md	要求大模型输出目录树、README、可运行 MVP 及标注 [扩展] 的接口占位。关联背景：doc/论文写作协作智能体开发项目计划.md。
多智能体工作流基础图结构	doc/代码框架要求.md（第一节 LangGraph Workflow）	含 Design→Think→Execute 链式节点、WorkflowState、节点包装器。
Agent / Tool / Memory / Router 抽象基类	doc/代码框架要求.md（第二至六节）	ReActAgent、PlanAndSolveAgent、BaseRouter 等 [扩展] 占位接口。
tests/ 单元测试文件	无独立 Prompt 存档（大模型辅助生成）	测试骨架由大模型辅助产出，成员后续补充用例、修正断言并用于回归验证。
LaTeX 子系统设计方案（非直接代码生成）	doc/TeX_Agent 核心系统设计方案： LaTeX 结构理解与智能诊断体系.md	作为 latex/ 子系统的设计依据；具体实现为成员自研。
Task/Plan Prompt 注入机制说明	doc/prompt_injection_map_task_plan.md	记录 task 与 plan 链路中 Prompt 拼接与注入位置（审计文档，非代码生成 Prompt）。

2.3 人工加工与演进说明

大模型仅提供可运行的最小骨架与扩展接口占位；项目的实际业务能力均由小组成员在此基础上人工开发完成（部分有大模型辅助，但后续提交的代码都经过开发人员人工审核和测试）：

1. **框架填充与扩展**：大模型最初生成的代码框架仅作示意，方便成员预先了解可能的接口，便于并行开发，后续开发过程中开发者会根据实际情况修改和扩展最初的框架（并同步给其他开发者）。在 `SimpleAgent`、`workflow/nodes.py`、`graph_builder.py` 等大模型骨架上，实现了条件边、并行分叉/汇聚、Web 自定义 workflow、Plan 模式动态构图等完整框架能力。
2. **业务功能自研**：LaTeX 诊断修复流水线、论文 checklist 审查、Ghost 幽灵窗口、Overleaf 编辑器、latex 论文写作、RAG 集成、Web 辅助工具等均为按需求独立开发。
3. **测试与纠错**：`tests/` 中大模型辅助生成的测试文件，经成员人工审阅后用于功能验证与辅助纠错，并非直接可交付的运行时代码。
4. **大模型辅助**：团队开发中仍会使用大模型进行辅助开发或辅助编写代码，但大模型生成的代码均会被人工第一时间审核和测试，确保代码质量和功能正确性。

2.4 按目录来源标注总表

目录/模块	来源	Prompt 溯源	人工加工要点
core/（消息/状态/异常）	大模型生成 + 自研	doc/代码框架要求.md	扩展 reducer、TeXAgentCLI 完整编排
agents/（基类骨架）	大模型生成 + 自研	doc/代码框架要求.md	实现 SimpleAgent、MultiSimpleAgent、ReflectionAgent
workflow/（基础图）	大模型生成 + 自研	doc/代码框架要求.md	条件边、并行节点、registry、run_dump
memory/（接口）	大模型生成 + 自研	doc/代码框架要求.md	BranchMemory、UserPersonaMemory 完整实现
router/	大模型生成 + 自研	doc/代码框架要求.md	AutoAgentsMASPlanner 完整实现
context/	大模型辅助 + 自研	—	GSSC 流水线、context_profiles 集成

目录/模块	来源	Prompt 溯源	人工加工要点
config/	大模型辅助 + 自研	—	上下文 Profile、25+ 工作流 JSON、Planner Schema
tools/（arxiv 骨架）	大模型生成 + 自研 + 复用开源	doc/代码框架要求.md	30+ 业务工具（PDF、checklist、LaTeX 等）
latex/	大模型辅助 + 自研	设计文档（见上表）	完整诊断修复、Ghost、Watch 服务
rag/	大模型辅助 + 自研 + 复用开源	—	ChromaDB 管道、CLI 工具
ui/	大模型辅助 + 自研	—	Web UI、Overleaf、Ghost 前端
cli/、utils/	大模型辅助 + 自研	—	命令框架、日志、取消控制等
check.py	自研	—	批量审查脚本
check_text.py			
tests/	大模型辅助 + 自研	—	补充用例、集成测试、夹具维护

2.5 第三方开源依赖清单

以下依赖来自 requirements.txt，按功能分组列出（版本约束以文件为准）。

2.5.1 工作流与 LLM 框架

包名	最低版本	本项目用途
langgraph	0.2.0	多智能体工作流图编排引擎
langchain	0.3.0	LLM 应用框架（辅助）
langchain-openai	0.2.0	OpenAI 兼容 API 接入
langchain-core	0.3.0	LangChain 核心抽象
pydantic	2.0.0	数据模型校验（WorkflowMessage、LaTeX 模型等）
python-dotenv	1.0.0	.env 环境变量加载

包名	最低版本	本项目用途
google-genai	1.0.0	Gemini 多模态 API
httpx	0.27.0	异步 HTTP 客户端
arxiv	2.1.0	arXiv 文献检索 API

2.5.2 Web 服务

包名	最低版本	本项目用途
fastapi	0.110.0	Web UI / Overleaf / Ghost HTTP 服务
uvicorn[standard]	0.27.0	ASGI 服务器
anyio	4.0.0	异步 I/O 兼容层
python-multipart	0.0.9	文件上传解析

2.5.3 RAG 与向量检索

包名	最低版本	本项目用途
chromadb	0.5.0	本地向量数据库（RAG 持久化）

2.5.4 LaTeX 诊断

包名	最低版本	本项目用途
pylatexenc	2.10	LaTeX 源码解析辅助
watchdog	4.0.0	目录文件监视（Ghost / Watch 服务）
系统级依赖（非 <i>pip</i> ）： <i>TeX Live</i> / <i>MiKTeX</i> 、 <i>latexmk</i> 、 <i>chktex</i>		

2.5.5 PDF 与文档解析

包名	最低版本	本项目用途
pymupdf	1.24.0	PDF 文本快速提取
pypdf	4.2.0	PDF 读取
pdfplumber	0.11.0	PDF 版面分析
PyPDF2	3.0.0	PDF 读取（兼容）
python-docx	1.1.0	Word 文档读取

包名	最低版本	本项目用途
docling	2.0	文档结构化解析（PDF→MD+JSON）

2.5.6 可视化、工具与测试

包名	最低版本	本项目用途
matplotlib	3.8.0	Web 工具箱图表绘制
qrcode[pil]	7.4.0	二维码生成
pytest	7.0.0	单元/集成测试框架
pytest-asyncio	0.21.0	异步测试支持

2.5.7 前端 vendor 开源库（非 pip）

文件	用途
ui/web/static/vendor/marked.min.js	Markdown 渲染
ui/web/static/vendor/purify.min.js	HTML 消毒（XSS 防护）
ui/web/static/vendor/highlight.min.js	代码语法高亮
ui/overleaf/static/vendor/pdf.min.js	PDF.js 预览

3 项目概述

TeX_Agent 是基于 **LangGraph + 多智能体（MAS）** 架构的学术论文写作智能辅助系统。系统覆盖文献检索、LaTeX 结构诊断与修复、论文写作和润色、论文合规审查（checklist）、RAG 知识检索、多分支对话上下文管理、工作流可视化编排等功能。

3.1 架构总览

- 入口层：start_all.bat（Windows 一键启动双 Web 服务）、main.py（CLI REPL）；ui/web/server.py（Chat Mode）、ui/overleaf/server.py（Writing Mode）；check.py、check_text.py（批量审查脚本）
- 编排层：core/agent_cli.py 的 TeXAgentCLI 串联记忆、分支、工作流
- 工作流引擎：workflow/ 将 JSON/TaskPlan 编译为 LangGraph 图并执行节点
- 智能体层：agents/ 提供 SimpleAgent、MultiSimpleAgent 等

- 工具层: tools/ 可插拔工具 (arXiv、PDF 解析、LaTeX 诊断、RAG 检索等)
- LaTeX 子系统: latex/ 项目索引、诊断修复、润色建议、LaTeX 写作、文件监视与幽灵窗口
- 支撑模块: config/、context/、memory/、rag/、router/

3.2 快速启动

环境配置完成后, Windows 下可双击根目录 `start_all.bat` 一键启动两个 Web 服务 (与 README.md 快速启动说明一致):

服务	地址	说明
Chat Mode	<code>http://127.0.0.1:8765/</code>	Web UI: workflow编排、论文审查、RAG 等
Writing Mode	<code>http://127.0.0.1:8772/</code>	Overleaf 风格在线 LaTeX 编辑器

脚本会先清理 8765/8772 端口上的旧进程, 再分别在独立控制台窗口启动 `python -m ui.web.server` 与 `python -m ui.overleaf.server`。亦可手动分别执行上述命令; Linux/macOS 可参考 `scripts/start-texagent-web.sh` 启动 Chat Mode。

3.3 文档范围说明

本文档基于仓库源码写作 (https://github.com/BCyyyyyyTZ/Tex_Agent.git), 排除以下内容:

- .gitignore 中忽略的路径 (如 .env、storage/、knowledge_base/、output/、memory_store/、logs/ 等)。
- doc/ 目录及其下所有文件 (部分在 prompt 溯源中提及)。
- test、tests 及其下测试文件, 测试部分另有测试文档进行详细说明。

4 根目录文件

根目录存放项目入口脚本、依赖清单、环境模板及少量开发辅助脚本。

4.1 核心入口与配置

`main.py`

CLI 主入口。解析 `task/plan/` 裸输入, 创建 `TeXAgentCLI` 并进入 REPL 循环。
关键函数: `parse_task_args()` (解析 `-wf` workflow 参数)、`main()`。

start_all.bat

Windows 一键启动脚本：清理 8765/8772 端口旧进程后，分别在独立控制台窗口启动 Chat Mode (`python -m ui.web.server, http://127.0.0.1:8765/`) 与 Writing Mode (`python -m ui.overleaf.server, http://127.0.0.1:8772/`)。关闭对应控制台窗口即可停止服务。详见 README.md。

overleaf.py

Overleaf 风格 LaTeX 编辑器快捷启动脚本，委托 `ui.overleaf.server.main()`。支持 `--host/--port` 环境变量注入。

overleaf.bat

Windows 批处理包装，用于一键启动 Overleaf 服务。

check.py

无 UI 批量执行多模态 checklist 审查 (`checklist_multi_v1/v2/v3`)。读取 `config/run_config.json`，约定目录 `files/input`、`files/checklist`、`files/output`、`files/checked`。

check_text.py

无 UI 批量执行文本类 checklist 审查（默认 `checklist_text_v3`）。接受结构化 JSON 路径输入，`preflight` 节点不调用 LLM。

requirements.txt

Python 依赖清单。

.env.example

环境变量模板（API Key、模型名、RAG 路径、Web 端口等）。实际配置复制为 `.env`（已 gitignore）。

pytest.ini

pytest 配置（标记、路径等）。

thesis-checklists.md

学位论文审查清单样例，供 checklist 工作流引用。

README.md

项目说明、快速开始、工作流使用指南。

Framework.md

系统框架设计说明文档。

4.2 开发辅助脚本（以下划线开头）

`_fix_encoding.py`

编码修复一次性脚本。

`_fix_workflow_paths.py`

批量修正 workflow JSON 中路径引用。

`_run_all_tests.py`

聚合运行测试套件。

`_run_checklist_test.py`

checklist workflow 手动测试脚本。

`_run_docling_user_test.py`

Docling 解析用户测试脚本。

`_run_eeg_test.py`

EEG 相关 PDF 测试脚本。

`_verify_v2.py`

v2 workflow 验证脚本。

4.3 其他根目录文件

`.gitignore`

Git 忽略规则（含 `.env`、`storage/`、`doc/code_note/` 等）。

`paper.aux`, `paper.fls`, `paper.fdb_latexmk`

LaTeX 编译辅助产物（本地测试用）。

5 agents/ —智能体实现层

5.1 目录总体作用

所有 Agent 继承 `BaseAgent`，对外统一 `run()` / `ainvoke()` / `reset()` 接口。 workflow 节点（`workflow/nodes.py`）按节点配置中的 `agent_type` 实例化对应 Agent 并调用。

5.2 文件说明

`__init__.py`

包入口，导出 `BaseAgent`、`SimpleAgent`、`SimpleAgent_new`。

base_agent.py

抽象基类与 LLM 客户端封装。

- AgentMemory / AgentMemoryItem: 运行时内存
- LlmClient: OpenAI 兼容 API 客户端, response(prompt, attachments)
- GeminiClient: Google Gemini 客户端, 支持并行上传文件
- BaseAgent: set_llm()、call_tool()、run() (抽象)、ainvoke()

simple_agent.py

当前主力 Agent: 纯 LLM 推理, 工具由 workflow 的 tool 节点处理。

- SimpleAgent.run(): 拼接历史消息、调用 LLM、返回 WorkflowMessage
- _build_history_messages()、_trim_history(): 有界历史管理
- _append_llm_trace(): 写入 logs/llm_interactions_trace.txt

multi_simple_agent.py

Gemini 多模态 Agent, 支持 PDF 附件上传与 generate_content()。关键: _upload_local_pdf()、_normalize_attachments()。

reflection_agent.py

反思迭代 Agent: Generate → Reflect → Refine, 最多 3 轮。内嵌 SimpleAgent 作为执行器。

specified_agents/paper_check_agent.py

论文 PDF 合规检查专用 Agent。

- PaperCheckAgent: 从 rules_path 加载检查规则注入 system prompt
- paper_check(): 解析 BEGIN [...] END JSON 问题列表, 调用 pdf_comment 工具标注

6 core/ 一核心基础设施

6.1 目录总体作用

定义工作流统一消息协议、LangGraph 全局状态 (含 reducer)、自定义异常体系, 以及 CLI/Web 共用的应用编排入口 TeXAgentCLI。

6.2 文件说明

message.py

统一通信协议。

- WorkflowMessage: 标准消息 (role、source_type、content、metadata、payload)
- ensure_message()、normalize_message_list(): 输入归一化
- NodeOutput: 节点结构化输出 (result、summary、confidence、status)
- ToolResult: 工具执行结果 (success、output、error)

state.py

LangGraph 工作流状态定义。

- WorkflowState(TypedDict): messages、current_node、input、output、error、metadata、retrieved_context
- Reducer: _last_wins、_last_nonempty、_first_error、_merge_metadata
- normalize_node_output(): 任意输出收敛为 NodeOutput dict

exceptions.py

分层异常体系: TexAgentError → AgentError、ToolError、WorkflowError、MemoryError、RouterError、SecurityError、ConfigError。

agent_cli.py

应用编排核心类 TeXAgentCLI。

- run_task(): 执行默认/指定 YAML 工作流
- run_plan_task(): Plan 模式 (MAS 规划 → 动态构图 → 执行)
- run_auto_task(): Auto 单轮直连模式
- _build_app_for_workflow(): 通过 build_app_from_workflow 构建 LangGraph app
- _execute_with_app(): 统一执行器 (装配状态、invoke/stream、回写上下文)
- switch_branch() / create_branch() / merge_branch(): 分支管理
- get_branch_chat_history_for_api() / get_branch_tree_for_api(): Web API 数据

7 config/ 一配置层

7.1 目录总体作用

集中管理所有可调参数：环境变量、LLM 模型、RAG 路径、上下文 Profile、Planner Schema、静态工作流 JSON 注册表、会议日历数据等。

7.2 Python 模块

`__init__.py`

导出全局单例 `settings`。

`settings.py`

全局配置 dataclass。Settings 从 `.env` > 环境变量 > 默认值加载。含 LLM 模型名、API Key、RAG 持久化目录、Web 端口等。

`context_profiles.py`

上下文配置唯一数据源。定义 4 种 Profile(LEGACY/PIPELINE/DIALOGUE/AUTO_SINGLE)、路由规则、意图识别、交付契约。关键函数：`build_context_config()`、`resolve_delivery_addon()`、`user_requests_long_form()`。

`context_settings.py`

从 Profile 加载并合并节点覆盖；拼 Prompt、过滤对话。关键：`NodeContextBehavior`、`load_context_config()`、`build_agent_prompt()`、`filter_messages_for_memory()`。

`planner_config.py`

Plan 模式 LLM Schema 与 JSON 解析契约。常量：`PLAN_OUTPUT_SCHEMA`、`SUPERVISOR_OUTPUT_SCHEMA`、`COMPLEXITY_AGENT_MAP`。函数：`parse_llm_json()`、`resolve_final_delivery_addon()`。

`agent_config.py`

三类 Agent 的 `system_prompt` 与 `temperature`：`DESIGN_AGENT_CONFIG`、`THINK_AGENT_CONFIG`、`EXECUTE_AGENT_CONFIG`。

`auto_config.py`

Auto 单轮模式标识：`AUTO_WORKFLOW_LABEL`、`AUTO_NODE_ID`。

`logging_config.py`

日志 dictConfig：`get_logging_config()`、`setup_logging()`。

7.3 JSON 与数据文件

`workflow_registry.json`

工作流名称 → JSON 文件路径映射 (`default`、`checklist_*`、`latex_*` 等 24 条)。

`workflow/*.json`

静态工作流图定义 (`nodes/edges`)，共 25 个文件，分类如下：

- 默认与演示: `workflow_default_dynamic`、`workflow_five_nodes_example`、`workflow_tool_linear_demo`、`workflow_user_feedback_demo`、`workflow_parallel_c`、`workflow_plan_parallel_example`
- 挑战/测试: `workflow_challenge_*`、`workflow_docling_user_test`、`workflow_file_an`、`workflow_arxiv_research_user`
- 论文审查 (`multi`): `workflow_checklist_multi_v1~v4`
- 论文审查(`text`): `workflow_checklist_text_v1~v3`、`workflow_checklist_annotate_v`
- 学位论文: `workflow_thesis_checklist_task`、`workflow_thesis_chapter_extract`
- LaTeX 诊断: `workflow_latex_diagnose_v0/v1`

`run_config.json` / `run_config.example.json`

批处理运行配置 (`checklist`、PDF 路径)，供 `check.py` 使用。

`run_config_text.example.json`

文本类审查批处理配置示例，供 `check_text.py` 使用。

`conferences/deadlines_2026.json`

学术会议投稿截止日期静态数据，供 Web 顶会日历使用。

`context/README.md`

上下文配置说明，指向 `context_profiles.py`。

8 context/ 一上下文管理

8.1 目录总体作用

为 Agent 节点提供统一上下文接口，实现 GSSC 流水线 (Gather / Select / Structure / Compress)，将对话历史、RAG 检索结果、metadata 链、长期记忆拼装为 LLM Prompt。

8.2 文件说明

`__init__.py`

导出 `BaseContext`、`ContextManager`。

`base.py`

上下文抽象基类 `BaseContext`: `save()`、`load()`、`clear()`、`build()`。

`context_manager.py`

默认内存上下文实现 `ContextManager`。

- `build()` 四阶段流程:
 1. Gather: 从 `state.messages`、`retrieved_context`、`memory` 检索
 2. Select: 按 `history_mode` (`full/minimal`)、`context_profile` 过滤
 3. Structure: XML 风格块 (`<context type='...'>`)
 4. Compress: 按 `max_tokens` 截断
- `format_metadata_chain_for_prompt()`: 格式化 `metadata` 链

`context_policy.py`

兼容层,重导出 `config.context_settings` 中的策略函数:`should_persona_file_write()`、`is_echo_task()` 等。

9 workflow/ — workflow 引擎

9.1 目录总体作用

workflow 引擎核心: 解析 JSON/YAML/TaskPlan 配置, 构建 `LangGraph` 图, 创建节点函数, 处理条件边与并行汇聚, 驱动整个 MAS 执行流程。

9.2 文件说明

`__init__.py`

对外 API:`build_app_from_workflow`、`build_dynamic_graph`、`ConditionExpr`、`YAMLWorkflowParser` 等。

`workflow_parser.py`

JSON/YAML/TaskPlan → 节点/边配置。

- `NodeConfig`、`EdgeConfig`: 节点与边数据类
- `YAMLWorkflowParser`: 解析静态 workflow JSON

- `_translate_plan_to_graph_config()`: TaskPlan 转图配置

`graph_builder.py`

LangGraph 构图主入口。

- `build_dynamic_graph()`: 从节点/边列表构建 StateGraph
- `build_app_from_workflow()`: 从 workflow 名或配置构建可 invoke 的 app
- `load_workflow_graph_config()`: 从 registry 加载图配置
- `_register_conditional_edges()`、`_build_agent_instance()`

`nodes.py`

节点工厂 (v2 增量 delta 写回)。

- `make_agent_node()`: Agent 节点
- `make_tool_node()`: 工具节点
- `make_user_node()`: HITL 人机反馈节点
- `make_parallel_fork_node()` / `make_parallel_join_node()`: 并行分叉/汇聚
- `_finalize_node_state()`: 节点执行后状态写回核心逻辑

`condition_evaluator.py`

条件边统一求值。

- `ConditionExpr`: 条件表达式数据类
- `evaluate_condition()`、`route_by_conditions()`

`parallel_merger.py`

并行分支汇聚策略。

- `JoinPolicy` (enum)、`BranchResult`、`MergedResult`
- `merge_parallel_results()`

`workflow_registry.py`

从 `workflow_registry.json` 加载 workflow。`WorkflowRegistry`: `list_workflows()`、`get_workflow_spec()`、`load_graph_config()`。

`run_dump.py`

单次运行节点 I/O 落盘 (调试用)。`create_run_output_dir()`、`write_node_trace()`。

9.3 节点类型

- agent — LLM Agent 推理
- tool — 工具调用（如 `arxiv_search`、`latex_project`）
- user — 人机交互（HITL）
- `parallel_fork` / `parallel_join` — 并行分叉与汇聚

10 router/ 一任务规划与路由

10.1 目录总体作用

Plan 模式下将用户任务分解为多 Agent 执行计划，并转为可编译的工作流图；同时预留自适应路由接口供未来扩展。

10.2 文件说明

`__init__.py`

导出 `BaseRouter`、`RouteDecision`、`MASPlanner`、`TaskPlan`。

`base_router.py`

自适应路由接口（预留/未完整实现）。

- `RouteDecision`: 路由决策 dataclass
- `BaseRouter(ABC):route()、evaluate_complexity()、register_agent()`

`planner.py`

多智能体规划器核心。

- `TaskPlan`: 执行计划 dataclass（含 `__edges__`、`__entry__` 特殊键）
- `MASPlanner(ABC)`: 规划器抽象基类
- `AutoAgentsMASPlanner`: 具体实现
 - `decompose(task)`: `PlanAgent` + `Supervisor` 多轮迭代生成计划
 - `assign(plan, available_agents)`: 为节点分配 Agent 类型
 - `validate(plan, results)`: 验证执行结果
 - `to_graph_config(plan)`: `TaskPlan` → (`NodeConfig`, `EdgeConfig`)
 - 内部: `_plan_agent_call()`、`_supervisor_call()`、`_apply_supervisor_revisions`

10.3 规划流程

Round 1: PlanAgent 生成 agents/edges → Round 2~N: Supervisor 审查修订 → 输出 TaskPlan, 由 workflow/graph_builder 编译执行。

11 tools/ 一可插拔工具层

11.1 目录总体作用

所有工具继承 BaseTool, 通过 tool_list.py 统一注册, 供 workflow tool 节点按名称实例化调用。工具与 Agent 解耦, 可选依赖缺失时单工具跳过。

11.2 基础设施

base_tool.py

工具抽象基类 BaseTool: run() / arun()。

tool_list.py

工具注册表与构图过滤。tool_list 字典、build_tools_for_graph_node()、_safe_instantiate()。

__init__.py

导出 BaseTool、ToolResult。

web_tool_utils.py

Web 工具箱输出路径辅助 (输出目录、唯一文件名)。

11.3 已注册工具 (workflow 可用)

文件	说明与关键类
arxiv_tool.py	arXiv HTTP API 论文检索。ArxivSearchTool、ArxivPaper。
pdf_comment_tool.py	PDF 批量高亮/批注。PdfCommentTool。
file_loading_tool.py	读取本地文件 (含 PDF 文本)。FileLoadingTool。
command_running_tool.py	执行 shell 命令。CommandRunningTool。
docling_tool.py	Docling 文档解析为 MD+JSON。DoclingParseTool。

文件	说明与关键类
docling_search_tool.py	在 Docling JSON 中本地搜索页码。DoclingSearchTool。
markdown_section_tool.py	按章节标题提取 Markdown 段落。MarkdownSectionTool。
pymupdf_parse_tool.py	PyMuPDF 快速 PDF 文本提取。PyMuPDFParseTool。
thesis_outline_extract_tool.py	按 PDF 目录提取大纲/章节正文。ThesisOutlineExtractTool。
thesis_chapter_route_tool.py	根据 outline 生成六路章节路由。ThesisChapterRouteTool。
checklist_prepare_tool.py	将 checklist 切分为 6 份审查包。ChecklistPrepareTool。
chapter_index_tool.py	Markdown 章节结构索引。ChapterIndexTool。
ref_checker_tool.py	参考文献规范性检查。RefCheckerTool。
references_slice_and_docling_tool.py	切参考文献页 + Docling 解析。
figure_ref_checker_tool.py	图/表/算法/公式交叉引用检查。FigureRefCheckerTool。
rag_retrieve_tool.py	向量知识库语义检索。RAGRetrieveTool。
register_inputs_tool.py	解析 workflow 输入 (PDF/checklist 路径)。RegisterInputsTool。
preflight_inputs_tool.py	预检 workflow 输入依赖是否就绪。PreflightInputsTool。
gemini_upload_pdf_tool.py	上传 PDF 到 Gemini Files API。GeminiUploadPdfTool。
offer_artifact_download_tool.py	Web UI 本地下载 token 登记。
latex_project_tool.py	扫描 LaTeX 项目构建 ProjectIndex。LatexProjectTool。
latex_parser.py	单文件语法检查/结构/AST。LaTeXParserTool。
chktex_tool.py	ChkTeX 静态诊断 (L1)。ChkTeXTool。
latexmk_tool.py	latexmk 试编译 + log 诊断 (L2)。LatexmkTool。
latex_slice_tool.py	按诊断 issue 切片源码上下文。LatexSliceTool。
latex_merge_tool.py	合并多源诊断 + 未定义引用。LatexMergeTool。

文件	说明与关键类
latex_report_tool.py	汇总诊断 workflow 终态 JSON。LatexReportTool。
latex_fix_prepare_tool.py	为 L3 修复 Agent 组装 prompt 批次。LatexFixPrepareTool。
latex_collect_suggestions_tool.py	解析 fix_agent 输出为 Suggestion。
user_persona_tools.py	入口节点用户画像写库工具族。UserPersonaGetTool、UserPersonaSetTool、build_user_persona_tools()。

11.4 未注册 / Web 独立工具

以下工具存在但未进入 tool_list，主要由 Web 工具箱 API (/api/drawing/*) 直接调用：

- concept_diagram_tool.py 一概念图生成
- chart_plot_tool.py 一图表绘制
- image_gen_tool.py 一图像生成（预留）
- visualization_tool.py 一学术图表（预留）
- palette_tool.py 一配色方案
- latex_table_tool.py 一LaTeX 表格模板
- paper_title_tool.py 一论文标题灵感
- text_stats_tool.py 一文本统计
- qrcode_tool.py 一二维码生成

12 latex/ —LaTeX 子系统

12.1 目录总体作用

LaTeX 结构理解与智能诊断修复子系统(阶段 0-10):数据契约、项目扫描、L1/L2/L3 诊断与修复流水线、源码切片、建议应用、文件监视与「幽灵窗口」HTTP 服务。与 tools/latex_* 工具配合形成完整闭环。

12.2 核心数据与契约

`models.py`

Pydantic 数据契约: `DiagnosticIssue`、`Suggestion`、`ProjectIndex`、`LabelDef`、`RefEntry`、`BibEntry` 等; `make_issue_id()`。

`constants.py`

枚举与 metadata 键: `Severity`、`IssueSource`、`METADATA_LATEX_*`。

`serialize.py`

JSON 序列化: `to_json()`、`from_json()`、`to_dict()`、`from_dict()`。

`paths.py`

路径规范化: `normalize_rel_path()`、`path_from_user_string()`、`resolve_tex_file()`。

`coerce_payload.py`

工具输入 JSON 解析: `coerce_json_payload()`、`parse_root_from_user_input()`。

12.3 项目扫描与索引

`project_index.py`

扫描.tex 文件图、checksum、main.tex。`build_project_index()`、`extract_inputs()`、`iter_tex_files()`。

`project_tree.py`

可视化项目树 (含 bib 边): `build_project_tree()`。

`bib_index.py`

.bib 解析与索引: `parse_bib_file()`、`build_bib_index()`、`enrich_bibliography()`。

`refs_index.py`

\ref/\cite 索引与未定义引用检测。`extract_refs_from_source()`、`collect_undefined_re`

`conventions_index.py`

导言区宏/包/版式约定: `parse_preamble()`、`build_conventions()`。

12.4 解析与语法检查

`single_file_parser.py`

单文件解析入口: `parse_tex_source()`、`parse_tex_file()`。

`syntax_check.py`

括号/环境平衡检查: `check_syntax()`、`check_brace_balance()`。

structure_extract.py

文档结构提取: `extract_structure()`。

ast_parse.py

LaTeX AST 简化: `parse_to_ast()`。

tex_source.py

注释剥离与读文件: `strip_comments()`、`read_tex_file()`。

12.5 诊断 (L1/L2) 与合并

chktex_runner.py / chktex_parser.py

调用 ChkTeX 并解析输出: `run_chktex()`、`parse_chktex_output()`。

latexmk_runner.py

调用 latexmk 试编译: `run_latexmk()`、`build_latexmk_argv()`。

log_parser.py

解析 LaTeX 编译日志: `parse_latex_log()`、`tail_log_text()`。

issues.py

多源诊断合并去重: `merge_issues()`、`merge_issue_lists()`。

diagnose_io.py

工具输出 ↔ 模型转换: `issues_from_tool_output()`、`project_from_tool_output()`。

dirty.py

文件脏状态检测 (checksum 变化): `compute_file_dirty()`。

12.6 切片、修复与建议

slice.py

按 issue 切片源码上下文: `slice_around_issue()`、`slice_issues()`。

fix_batch.py

L3 修复批次组装: `select_error_issues()`、`build_fix_batch()`。

prompt_builder.py

修复 prompt 构建: `build_fix_prompt()`、`collect_ref_context_snippets()`。

fix_agent_prompt.py

L3 fix_agent 系统提示常量 `LATEX_FIX_AGENT_SYSTEM_PROMPT`。

`polish_prompt.py` / `ghost_polish_prompt.py`

润色 prompt 构建（含幽灵窗口专用）。

`suggestion.py`

LLM 输出 → `Suggestion.parse_llm_suggestion_json()`、`parse_llm_suggestions_from_a`

`apply_edit.py`

将建议应用到.tex: `apply_suggestion_to_file()`、`offset_from_position()`。

`apply_compare.py`

对比模式应用建议（保留旧文为注释）：`apply_suggestion_compare_to_file()`。

12.7 监视与幽灵窗口

`watch_events.py`

监视事件模型：WatchSnapshot、WatchEvent。

`watch_service.py`

目录监视 + 自动诊断/修复：LatexWatchHandler、WatchService。

`ghost_watch_policy.py`

幽灵窗口监视策略（静默窗口、跳过 LLM）：GhostWatchPolicy。

`ghost_server.py`

FastAPI 幽灵窗口 HTTP 服务：`create_ghost_app()`、`run_ghost_server()`。

`ghost_cli.py` / `watch_cli.py`

幽灵服务与监视 CLI 入口：main()。

12.8 环境与导出

`tex_env.py`

TeX 环境探测：`probe_tex_env()`、TexEnvStatus。

`__init__.py`

公共 API 聚合导出（models、index、slice 等）。

12.9 LaTeX 诊断修复流水线

项目扫描 → L1(ChkTeX) + L2(latexmk) + 语法检查 → 合并去重 → 切片 → L3 LLM
修复 → 应用建议

13 rag/ 一检索增强生成

13.1 目录总体作用

RAG 模块: 文档加载、分块 → ChromaDB 向量化 → 语义检索, 供 RAGRetrieveTool 与 workflow retrieve 节点使用。抽象接口便于 Mock 与替换实现。

13.2 文件说明

`base_retriever.py`

抽象接口: RetrievedDocument、BaseRetriever、BaseRAGPipeline。

`vector_store.py`

ChromaDB 实现 ChromaRetriever.add_documents()、retrieve()、delete_by_ids()、delete_by_source()、list_stored_page()。

`document_loader.py`

文本分块与文件加载。chunk_text()、load_text_file()、load_and_chunk()。

`rag_pipeline.py`

端到端管道 RAGPipeline.index_text()、index_file()、retrieve()、clear()。

`document_parse.py`

Docling 文档解析(PDF 大文件分块等)。parse_document_to_dir()、resolve_source_path()

`store_listing.py`

向量库分页列举数据结构。StoredChunkRecord、format_stored_chunks_page()。

`rag_index_cli.py`

索引 CLI: resolve_user_file_path()、main()。

`rag_list_cli.py`

列举向量库 CLI (交互翻页)。

`rag_delete_cli.py`

按 id/source 删除或清空向量库 CLI。

`__init__.py`

导出 RAGPipeline、ChromaRetriever 等公共接口。

14 memory/ — 记忆系统

14.1 目录总体作用

Agent 记忆系统与用户画像持久化：支持共享/私有、可选分支、JSONL 落盘。UserPersonaMemory 供入口节点工具与 workflow prompt 注入用户偏好。

14.2 文件说明

base_memory.py

记忆抽象基类 BaseMemory。MemoryType 枚举；save()、load()、search()、clear()、get_size()。

simple_memory.py

基础内存实现 SimpleMemory，含混合检索评分。_score_query_doc()（关键词/Jaccard/新近性）、_tokenize()。

branch_memory.py

分支记忆 + JSONL 持久化。BranchMixin:create_branch()、switch_branch()、merge_to_main()。BranchMemory：结合分支与简单记忆的完整实现。

factory.py

记忆工厂 MemoryFactory.create_memory()、create_shared_memory()、create_private_memory()、create_hybrid_memory()。

persona_memory.py

线程安全用户画像 JSON 存储 UserPersonaMemory.apply_persona_memory_update()、format_for_prompt()、get_shared_user_persona_memory()。

__init__.py

导出 BaseMemory、SimpleMemory、BranchMemory、MemoryFactory、UserPersonaMemory。

15 ui/ — 用户界面层

15.1 目录总体作用

所有用户界面：主 Web 聊天 UI（FastAPI + 静态页）、Overleaf 风格 LaTeX 编辑器、LaTeX「幽灵窗口」浮层。负责启动后自动开浏览器、文件上传存储、工作流可视化编排等。

15.2 ui/web/ — 主 Web 服务

server.py

主 Web 服务（FastAPI，默认端口 8765）。

- TeXAgentWebUI：继承 TeXAgentCLI，Web 模式编排
- create_app()：FastAPI 应用工厂
- 核心 API: POST /api/chat (Task/Plan/流式)、GET/POST /api/branches、GET/PUT /api/workflow/draft、GET/POST /api/storage/pdfs、/api/rag/*、/api/latex/watch*、/api/drawing/*

ide_launch.py

服务启动后自动打开聊天页。schedule_open_ide()、open_ide_window()、default_open_ch

browser_open.py

跨平台打开 HTTP 链接（WSL 优先走 Windows 侧）：open_http_url()。

file_storage.py

上传文件按类别存入 storage/{pdfs,skills,checklists,documents}/。category_dir()、unique_stored_path()、resolve_safe_path()。

pdf_storage.py

旧 PDF 导入兼容层，委托 file_storage。

conferences_data.py

顶会投稿日历：load_conferences_config()、list_deadlines()。

15.3 ui/web/static/ — 主 Web 前端

文件	说明
index.html	主页面骨架（RAG/工作流/分支/聊天/资料五栏布局）
app.js	核心聊天逻辑：/api/chat 流式、Markdown 渲染、模式切换
workflow_builder.js	左侧工作流 DAG 编辑器，读写 /api/workflow/draft
branch_graph.js	对话分支 SVG 树，调用 /api/branches
rag_assets.js	RAG 索引/查询/记录管理 UI
web_assets.js	文件上传面板，维护 active_* 供聊天引用

文件	说明
drawing_tools.js	独立工具箱弹窗（图表、LaTeX 表格、配色、QR 等）
conferences_calendar.js	顶会日历浮层
user_settings.js	主题/用户偏好（localStorage）
panel_toggles.js	侧栏显隐控制
layout_resizer.js	五列布局拖拽调宽
styles.css 等	全局与组件样式
vendor/*.min.js	第三方库（marked、DOMPurify、highlight）

15.4 ui/overleaf/ — LaTeX 编辑器

server.py

Overleaf 风格 LaTeX 编辑器服务（FastAPI，默认端口 8772）：项目/模板 CRUD、编译 PDF、Agent 润色、论文生成。create_app()、_scan_files()、PolishBody、GenerateBody。

thesis_generator.py

用 SimpleAgent 根据大纲生成英文 thesis main.tex。generate_thesis_project()、_build_generation_prompt()。

static/

编辑器前端：index.html（项目列表）、editor.html/js/css（源码 + PDF 预览）、vendor/pdf.min.js（PDF.js）。

15.5 ui/ghost/ — 幽灵窗口

index.html

LaTeX Watch 实时建议/修复卡片浮层页面。

ghost.js

轮询 Watch API、展示行级建议、应用修复。

ghost.css

幽灵窗口样式。

16 cli/ —终端命令框架

16.1 目录总体作用

终端 REPL 的命令模式框架:命令注册、解析、分支管理与任务执行。由 `core/agent_cli.py` 的 `TeXAgentCLI` 引用。

16.2 文件说明

`commands.py`

命令抽象与注册表。

- `Command(ABC)`: 命令抽象基类, `execute()`、`get_help()`
- `CommandRegistry:register()`、`find_command()`、`execute()`、`show_help()`

`branch_commands.py`

对话分支 CLI 命令。`ListBranchesCommand`、`CreateBranchCommand`、`SwitchBranchCommand`、`MergeBranchCommand`、`ShowBranchCommand`。`BRANCH_COMMANDS`、`BRANCH_ALIASES`。

`task_commands.py`

任务/系统 CLI 命令。`TaskCommand` (解析 `-wf/-b` 参数)、`StatusCommand`、`ClearCommand`、`ExitCommand`、`HelpCommand`。`get_task_commands()`。

17 utils/ —通用工具

17.1 目录总体作用

跨模块通用工具:日志、异步、终端输出、回复格式化、协作取消、Web 下载令牌、PDF 章节提取、Unicode 规范化等。

17.2 文件说明

`__init__.py`

导出 `get_logger`、`run_async`、`gather_with_timeout`、`run_with_semaphore`。

`logger.py`

统一日志 (首次调用初始化 `config/logging_config`): `get_logger(name)`。

`concurrency.py`

同步上下文跑协程、并发控制。`run_async()`、`gather_with_timeout()`、`run_with_semaphore`

display.py

CLI 输出格式化 (含 Windows emoji 降级)。safe_print()、DisplayFormatter (banner、separator、print_result)。

loading_spinner.py

终端加载动画: LoadingSpinner、with_loading()、run_with_loading()。

run_cancel.py

Ctrl+C 协作式取消 (Web 流式 + arXiv 等长阻塞)。request_run_cancel()、is_run_cancelled()、interruptible_sleep()、install_sigint_handler()。

reply_format.py

Web 回复 Markdown 整理 (去 cite 垃圾、合并碎段)。strip_citation_artifacts()、normalize_reply_display()、format_reply_for_ui()。

web_artifact_registry.py

一次性下载 token (/api/download/artifact?token=)。register_file()、get_file_path()。

thesis_pdf_extract.py

基于 PDF 书签的学位论文按章提取 → Markdown + Docling JSON。run_thesis_pdf_extract()、select_nodes_by_chapters()、build_outline_tree()。

text_normalize.py

Unicode NFKC + 康熙部首补丁 (Docling/OCR 乱码修复)。normalize()、normalize_for_search()。

utils.py

嵌套 dict 赋值: set_nested_value(d, keys, value)。

18 scripts/ 一启动脚本

18.1 目录总体作用

Web UI 启动脚本: 子进程运行 python -m ui.web.server (Chat Mode, 默认 8765), 可选自动打开浏览器。Writing Mode (ui.overleaf.server, 默认 8772) 需单独启动, 或由根目录 start_all.bat 一并拉起。

18.2 文件说明

start_texagent_web.py

Python 启动器。解析 --host/--port/--no-browser, SIGINT 时终止子进程。

关键函数：`main()`。

`start-texagent-web.sh`

Shell 包装：`cd` 到仓库根目录后 `exec` Python 脚本。适用于 Linux/macOS/WSL。